



AI 301 @ Workplace 2026

Henrique Sanchez
2026-09-14

Recap...

Where we've been:

- **AI 101:** What LLMs are, how to start prompting, what to watch out for
- **AI 201:** How LLMs think, structured prompting, workflows, task decomposition, quality control

Where you are now:

- You've been using AI tools for weeks or months
- You've had wins and frustrations
- You've felt the ceiling of single-session prompting

The Past, Present, and (Near) Future...

When	Stage	What It Means	What You Can Do	
Early 2023	Basic Chatbots	AI answers questions from memory	Ask questions, get text answers (often wrong)	} AI as a tool
Mid 2023	Document Attachments	AI can read your files	"Summarize this PDF" or "Analyze this spreadsheet"	
Late 2023	Custom Bots + Knowledge Bases	AI trained on your specific docs	Build a "company expert" that knows internal policies	
2024	Tool-Using Agents	AI can take actions, not just talk	Autonomously send emails, query databases, call APIs	
Early 2025	Looping Agents	AI works autonomously in cycles	"Build this feature" → AI codes, tests, fixes, repeats until it's done, navigating obstacles	} AI as a colleague
Late 2025–2026	Orchestrators & Swarms	Multiple specialized AIs coordinate	Teams of agents: one plans, one researches, one executes	

Most are still here!



What we'll cover today

1. **Custom Bots & Knowledge Bases** (Leveraging your own data)
2. **Context Management** (How to manage memory and avoid the traps)
3. **From Chatbot to Agent** (What actually changes)
4. **The Information Map** (Five types of knowledge work AI can transform)
5. **Failure Patterns** (The ways a chatbot fails even when a prompt is good)
6. **The Eval Mindset** (How to know an output is wrong before it costs you)

Don't worry about memorizing anything, all materials and examples will be available in the description below. Focus on following the logic and understanding the *hows* and the *whys*

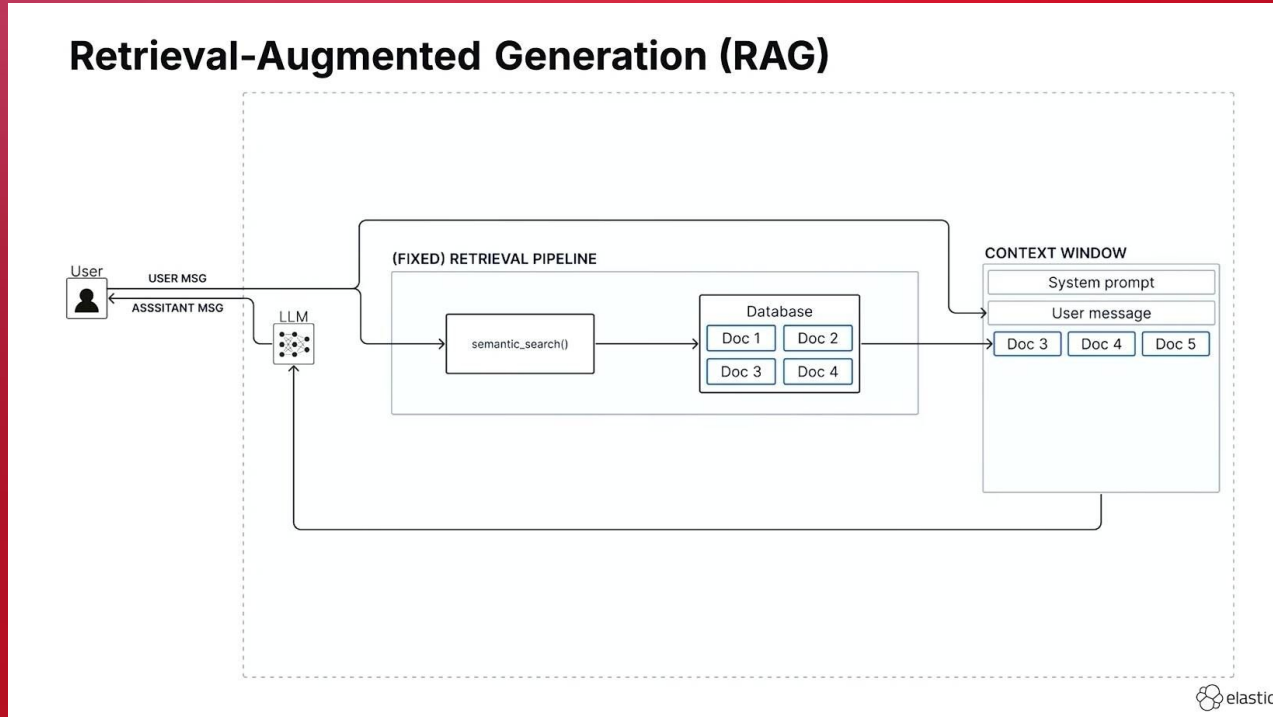
1. Custom Bots & Knowledge Bases

A **custom** or **personalized bot** (which enables a knowledge base) is essentially an LLM which is fed or has access to a selection of **files of your choice** prior to any conversation starting, supplementing its pre-existing training data towards more relevant usecases.

This primarily **saves** the user **time** and effort from manually uploading all relevant documents in each new conversation/instance/session, and grounds the output towards more truthful replies, reducing confabulations/hallucinations.

1. Custom Bots & Knowledge Bases

This is accomplished primarily through a mechanism/process called **Retrieval-Augmented Generation (RAG)**



1. Custom Bots & Knowledge Bases

With custom bots and knowledge bases, not only should we consider how we prompt the model in our chat, and what documents it should have access to via RAG, but also the custom instructions (known as **system prompt**) it should follow everytime that are independent of the present task we want to do.

While RAG provides the dynamic facts (what the bot knows) and your chat prompt provides the immediate task (what the bot should do right now), the system prompt **defines how the bot behaves, speaks, and enforces constraints across every single session.**

1. Custom Bots & Knowledge Bases

Examples of system prompts:

- *The Grounded Analyst:* You are a data assistant for quarterly reporting. Always trace every metric back to the specific document and page number provided in the knowledge base. If the retrieved context does not contain the answer, state that clearly rather than estimating or filling in the gaps.
- *The Policy Enforcer:* You are an internal compliance assistant. Answer questions strictly using the uploaded company policy documents. Format your answers as short bullet points, and always append a reminder to consult the legal team for official interpretations.
- *The Tone and Format Guide:* You are a senior project manager reviewer. Review the provided draft status update using a direct, concise, and constructive tone. Eliminate all corporate buzzwords, passive voice, and unnecessary introductory pleasantries.

1. Custom Bots & Knowledge Bases

To maximize the usefulness of custom bots:

- Prepare documents with clear headings and consistent formats;
- Put the stable, always-true material first and the changing material last;
- Write the custom instructions once, carefully, because every conversation inherits them;
- Verify against the source, because a knowledge base retrieves *chunks*, and a chunk can be the wrong one.

1. Custom Bots & Knowledge Bases

Example **Unstructured**:

The plant manager, Bob, started our aeration journey back in 2005. We generally try to keep the blowers running when the dissolved oxygen level in the main basin drops below 2.0 mg/L, but if it is really hot outside, we sometimes adjust that to 2.5 because the bacteria get stressed. Just make sure you do not turn them off completely if the sensors are acting up, unless it is a total power failure.

(RAG-based system *will* **struggle**)

You can use AI to
do the
structuring!



Example **Structured**:

SECTION: Aeration Control (Dissolved Oxygen)
CONSTRAINT 1: Target DO Range = 2.0 to 2.5 mg/L.
CONSTRAINT 2: Do not disable blowers during operational hours.
CONSTRAINT 3: Ambient Temperature $\geq 30^{\circ}\text{C}$ requires 2.5 mg/L setpoint.
PROCEDURE:
1. Verify current DO level via sensor output.
2. If $\text{DO} < 2.0 \text{ mg/L}$, activate blower Stage 1.
3. If ambient temperature $\geq 30^{\circ}\text{C}$, increase target to 2.5 mg/L.

(RAG-based system *will* **thrive**)

1. Custom Bots & Knowledge Bases

Some ready-to-use examples are:

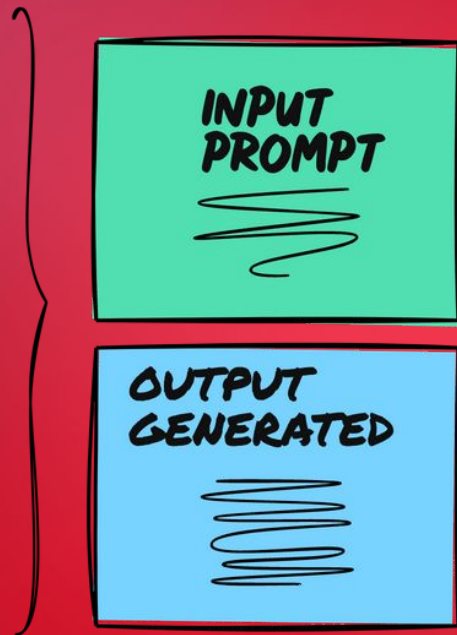
- NotebookLM
- GPTs in ChatGPT
- Projects/Artifacts in Claude
- Any agentic tool also searches directories via semantic search and RAGs

As **agents** become more adopted and their tools mature, features like **agentic search** are expected to gradually replace the simpler RAG-based solutions for custom bots and knowledge bases.

However, the matter of managing all this information as context remains.

2. Context Management

CONTEXT
WINDOW



The context window is the amount of information an LLM can "see" at once.

As you chat back and forth with a chatbot, context doesn't just sit there. **It accumulates.**

2. Context Management

Every prompt you send adds text. Every retrieved document chunk adds tokens. As the chat history fills, the bot's attention gets diluted. Early instructions get forgotten. Noise drowns signal. By the end of a long, complex discussion, the chatbot is often solving a subtly different problem than the one you assigned at the start.

This is called **context degradation**, and it is the most common way your chatbot interactions start to lose quality.

The default assumption, that you should keep everything in one long, infinite chat thread to preserve continuity, is *wrong*.

Instead, a more useful assumption is that context must be **computed**, not accumulated.

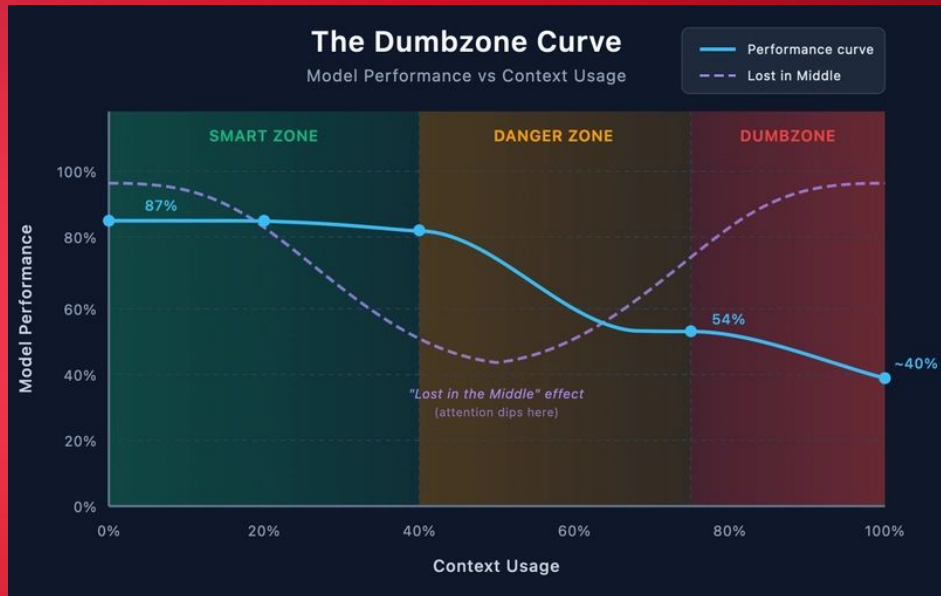
Therefore, a chat session is not a diary you add to indefinitely. Instead, it is a working memory you **actively manage**.

2. Context Management

How to know when the context window is in the danger zone?

Rules of thumbs/heuristics:

- Context degradation happens much earlier than the model's maximum capacity. **A safe area to operate is to stay under the smaller of ~120k tokens or ~50% of max context.**
- Generally speaking between **10 to 20 turns** of chatting with the LLM it might be a good idea to start a new instance
- If your LLM says **"You are absolutely right..."** or a variation of this after you point out a mistake or pushback on a concept, it is probably time to start a new conversation elsewhere.



2. Context Management

LLMs have no internal state, meaning that every time a user presses Enter for their prompt, the LLM processes their request *from scratch*, with the request appended to the (previous) *conversation array*. The chatbot interface only gives the **illusion** of continuity.

This is a superpower, not a limitation! We can “time-travel” and explore different trajectories in the conversation without degradation as a result!

2. Context Management

How to manage the context (aka how to “time travel”)?

Since the model is stateless, a conversation is just an array of text, and text can be edited, duplicated, and thrown away. The chatbot interface hides this, but the powers are already in your hands:

- **Rewind.** Most chatbots let you edit your own earlier message and resend. Don't argue forward with a degraded thread ("no, I meant...") since every correction adds noise. Go back and make the original prompt right. Everything after it never happened. (No edit button? Copy the thread up to that point into a new chat. Same move).
- **Parallelize.** Same prompt, three fresh tabs, side by side. New instances are free, so sample more than one future, then take the best answer or merge the good parts.

Backwards or sideways, it's all the same operation: **choosing which branch of the conversation to continue.**

2. Context Management

How to manage the context (aka how to “time travel”)?

Time travel lets you recover from degradation. The stronger alternative is to *prevent it* altogether:

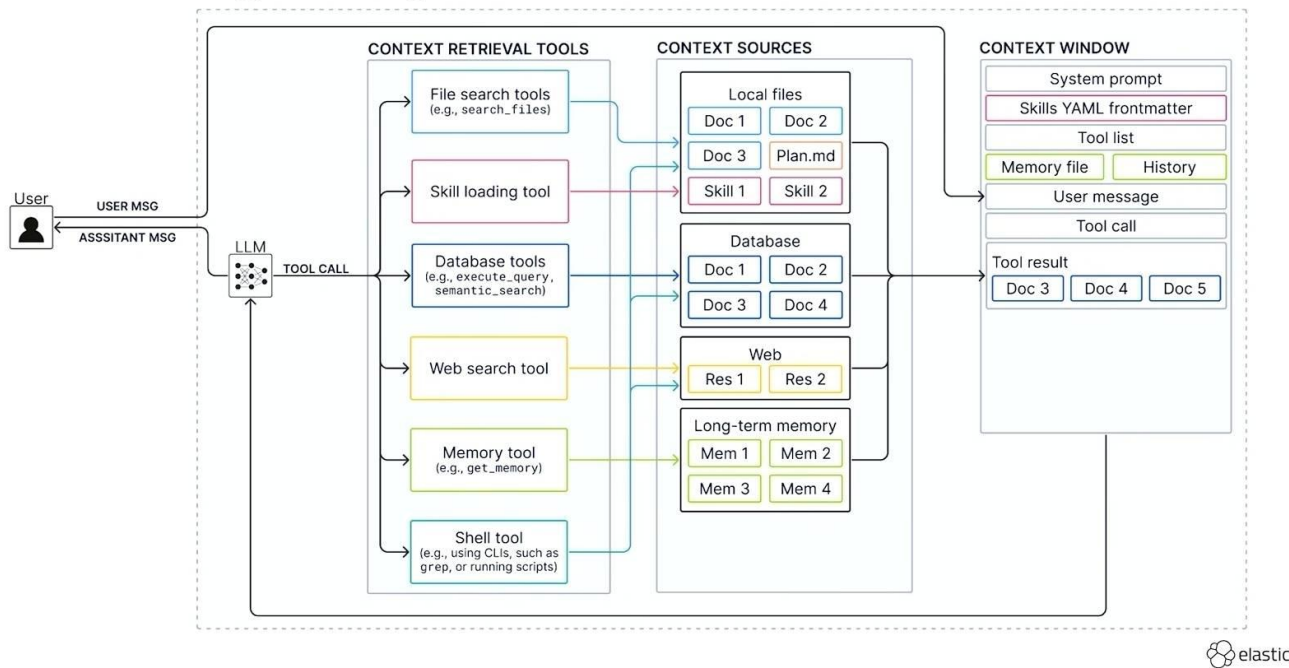
- **Handoff.** When a thread gets long and heavy, ask the bot to compress itself:
"Summarize what we've established, such as decisions, constraints, open questions, and assume a new fresh instance of you will read it "Then continue in a new chat with only that summary. The new instance inherits your conclusions, and not your noise (although this also has its own limitations...)"
- **Externalize.** Keep that summary in a document you own, and paste it into each new chat. The chat is a whiteboard, and the doc is the memory.

Rule of thumb: restart beats repair.

Every move mentioned here (rewinding, forking, parallel sessions, compression, memory) can be automated. Hold that thought, as that's what agents can do.

2. Context Management

Context Engineering



Context management is therefore adding the right **type** of information and the right **amount** of information at the right **time/step** of the **process/workflow**

3. From Chatbot to Agent

Wrong mental model: An agent is a smarter chatbot that can do more things

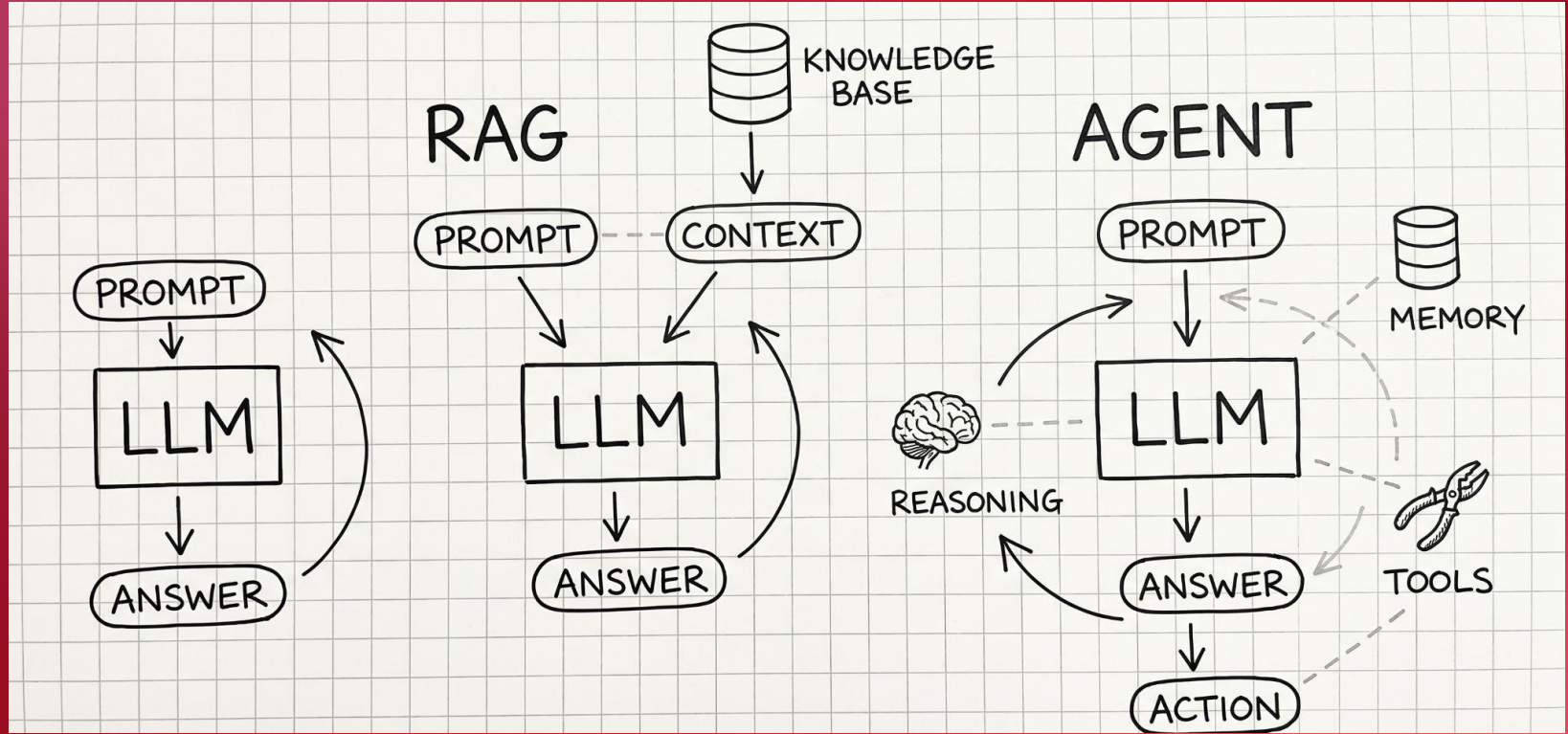
Right mental model: An agent is an autonomous execution loop that observes, decides, acts, and adjusts, without waiting for you to tell it what to do next

Three defining capabilities that make something an agent:

- **Tools:** It can take actions beyond generating text
- **Loop:** It can run in cycles, like propose an action, execute it, observe the result, and decide what to do next
- **State:** It can maintain memory and context across steps, so it isn't starting from zero with every exchange

In short: A chatbot answers, an agent acts.

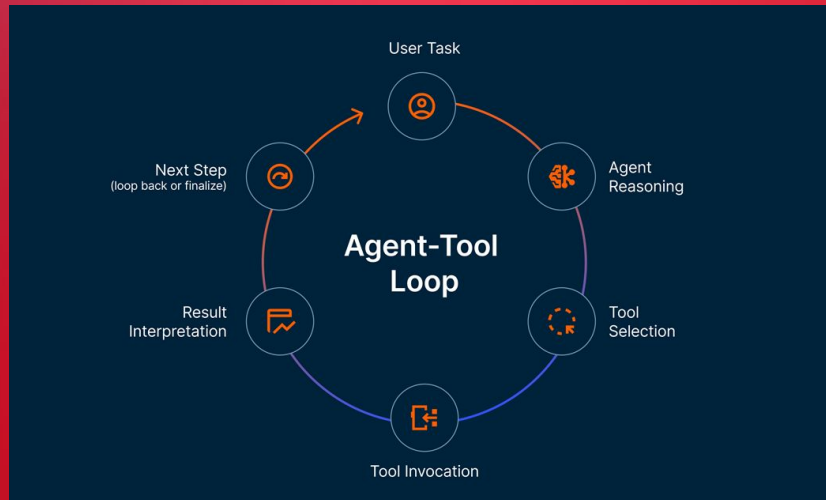
3. From Chatbot to Agent



3. From Chatbot to Agent

As years have passed, many of the previous simple chatbots have quietly transformed to agents under the hood, even though the chat interface appears the same.

Therefore, the AI you are interacting with no longer just returns tokens from its training data, but can do other actions, including: **read/write files, web search, running code (script execution), and connect APIs to external systems**



3. From Chatbot to Agent

So what is an agent, really?

Not a smarter model. An agent is the *same LLM you have been using all along*, wrapped in a **harness** that gives it **tools** and lets it run in a **loop** until the job is done.

Which means every context management move from the last section, it can now do to itself:

- **Rewind**: it retries its own failures from an earlier checkpoint instead of pushing a bad result downstream
- **Parallelize**: it can run several attempts side by side and keep the best one
- **Externalize**: it keeps its working notes in files, so its memory survives the session, and you can inspect it

You set the goal and the guardrails once. It manages its own working memory while it works, for minutes or hours, without waiting for you to press Enter.

One consequence of this is that with the same model, we also have the same failure modes, but now running unsupervised. Hold that thought for Section 5.

3. From Chatbot to Agent

The most useful mental model for working with an agent:

"An extremely capable, completely literal-minded colleague who executes exactly what you specify, at superhuman speed, with no common sense about what you actually meant."

What this means in practice:

- If you tell it to "clean up the folder" and it will, including the files you didn't mean it to touch
- Did you forget to mention a constraint? The agent will not infer the constraint
- Did it reach the edge of your instructions? It makes its best guess and keeps going, rather than stopping

The upside however, is that a well-specified task gets executed unfailingly, at speed, without complaining, and without forgetting a step.

The implication here is that your job is to be a better specifier, not a better prompter.

Which raises the real question: **what** should you be specifying? That is the Information Map.

4. The Information Map

The best use of agents is for knowledge work that involves **moving, transforming, or synthesizing information**.

Most knowledge work is exactly this. The question is whether you can see it clearly enough to describe it to something that will take you literally.

Before you can design an agent workflow, you need to answer:

- What **type** of information am I working with?
- What **transformation** does it need?
- What does "**done**" look like?

These are not obvious questions. Most knowledge workers have never needed to articulate the structure of their own work this precisely. Agents force the clarity that human colleagues never required.

The reward: Once you can describe a transformation precisely, it becomes repeatable. Not just faster, but *repeatable*.

4. The Information Map

Every task you might want to hand to an agent can be broken into five atomic types. Most real workflows combine several:

Type	What it is	Example
Extraction	Pulling specific information from unstructured sources	Action items from meeting notes; risk clauses from contracts
Synthesis	Combining multiple inputs into a coherent whole	Themes across 50 customer interviews; quarterly narrative from 12 reports
Analysis	Applying logic, rules, or calculation to structured problems	Compliance gap assessment; financial ratio evaluation
Generation	Creating new content meeting specific requirements	First draft of a proposal; email response maintaining voice
Validation	Checking correctness against defined rules	Verifying calculations; catching inconsistencies; adversarial review

4. The Information Map

The practical exercise:

Take one recurring task you do regularly, something that happens weekly or monthly, that you often find tedious or time-consuming. Now ask:

- What type of information **comes in**? (Documents, emails, data, meeting notes, reports?)
- What **transformation** does it need? (Which of the five types?)
- What does the **output** look like? (Format, length, audience?)
- Where does your **judgment** actually matter? (The part only you can do?)

Most of what feels like "knowledge work" is actually a sequence of extractions and syntheses with a thin layer of human judgment applied at the end.

That thin layer is where your value lives. The sequence before it is where the agent can work.

5. Failure Patterns

In **AI 101** we covered the failures you can see: hallucinations, false confidence, slop.

In **AI 201** we gave you structure to reduce them: the six-element prompt, verification steps, source grounding.

In **Section 2** we covered the failure that comes from memory: *context degradation*.

This section is about a different family. These failures happen with a good prompt, a fresh chat, and a capable model. Structure does not prevent them, because they don't come from the output or the memory. They come from the **interaction**: from what you say, how you say it, and how you and the model hand things back and forth.

There are five. They all come from one fact you already know.

5. Failure Patterns

The model *continues the text it is given*. All of it. Your framing, your constraints, the document you pasted, the answer from the previous step, its own fluent prose from three turns ago. It has **no separate channel for "what is true"**. Everything in the conversation is equally real to it.

Hold that in mind and each pattern below becomes obvious rather than surprising:

Pattern	In one line
1. Premise inheritance	It builds on your mistakes
2. Question substitution	It answers the nearby, easier question
3. Literal compliance	It satisfies to the letter and loses the point
4. Error laundering	Mistakes get cleaner with every step of a chain
5. Fluent wrongness	The wrong answer looks identical to the right one

5. Failure Patterns: Premise Inheritance

What it looks like: "Our conversion rate is 12%, which is above industry average. Analyze why we're outperforming." The real rate is 1.2%. You receive a confident, well-structured analysis of a success that never happened.

Why it happens: your message is part of the text it continues. A confident premise is most likely to be continued by agreement. It is not lying to you; it is *completing your pattern*.

The tell: it has not disagreed with you once in the entire conversation.

How to catch it:

- Separate the *data* from your *interpretation*.
- Paste the numbers first, and ask what it notices, *including anything that looks off*, before you say what you think they mean.
- Ask directly: "What in my framing would you push back on if you were a skeptical colleague?"
- Once in a while, plant a known error in a prompt and see if it gets caught. If it doesn't, you have just learned how your chatbot treats you when you are wrong.

5. Failure Patterns: Question Substitution

What it looks like: you ask *which of these three vendors fits our constraints*, and get a *comparison of three vendors*. You ask for the risks of **our** plan and get the risks of **plans like ours**. You ask whether a clause is unusual *for a French supplier*, and get a general explanation of the clause.

Why it happens: the most likely continuation of a question is the answer to the **most common version** of that question. Your specific question is rarer than its generic cousin, so the generic cousin wins. (Humans do this too)

The tell: the answer would work just as well for someone else's situation. Nothing in it could *only* have been written for you.

How to catch it: before it does the work, make it say the work back:

"Before you start: restate in one paragraph what I'm asking for, what you are NOT going to do, and what you would need to check"

It costs one turn. You see the swap before you pay for it.

5. Failure Patterns: Literal Compliance

What it looks like: "Keep it under 200 words" and the caveat that was the whole point is gone. "Make it sound more confident" and the uncertainty that was *true* gets deleted. "Remove the jargon" and the precision goes with it. "Make the numbers add up" and a number changes.

Why it happens: it optimizes the **constraint you wrote**, not the **goal you had**. Constraints are text; goals are in your head. Whatever is measurable in your words wins over whatever isn't.

The tell: the constraint is perfectly satisfied and the result is worse. Something you cared about quietly disappeared to make room.

How to catch it:

- Attach a **why** to every constraint. "Under 200 words *because it's a google chat message*, and the risk caveat must survive."
- After any constrained rewrite, ask: "What did you remove or weaken to satisfy that?"
- Say what must **survive**, not only what must go.

5. Failure Patterns: Error Laundering

What it looks like: in 201 you learned to chain: *extract* → *analyze* → *synthesize*. Step 1 misreads one number in a table. Step 2 builds three trends on it. Step 3 writes an executive summary with the wrong number as its headline, properly formatted and confidently phrased. Nobody in the chain made anything up. The number just got **better dressed** at every step.

Why it happens: whatever you paste into the next chat is treated as ground truth. The model cannot tell a fact from a previous step's guess. Both are just text. Each step **adds polish and removes provenance**.

The tell: the final output cites "the analysis" instead of the source. You can't trace a claim back to a document in under a minute.

How to catch it:

- Verify at the **cheapest point**: the extraction, not the summary. A wrong number is easy to spot in a table and nearly invisible in a paragraph.
- Keep **sources attached to claims** all the way through the chain. (201's source grounding technique, and now you know what it is protecting you from)

5. Failure Patterns: Fluent Wrongness

What it looks like: the wrong answer and the right answer are **indistinguishable**. Same tone, same formatting, same confidence. No error message, no hedge, no signal.

Why it happens: fluency is what the model is trained to produce. Correctness happens when fluency and truth coincide, which is often, but the polish is the same either way.

Two things make this worse, and both are about *you*:

- **The polish trap.** The better an output looks, the less carefully you read it. Your attention drops while the risk stays constant.
- **The expertise gap.** Inside your field you catch errors by reading. Outside it you don't, and you *can't feel the difference*. (Michael Crichton called this **Gell-Mann amnesia**: you spot the errors in the article about your own field, turn the page, and trust the next one). So the errors that survive are concentrated exactly where you are least able to tell.

The tell: there isn't one. That is the point.

How to catch it: you can't catch it by reading better. You catch it with a **check that does not depend on how the output looks**. That is the next section.

5. Failure Patterns

Notice what protects you from all five right now: **you are between every turn**. You read before you paste. You press Enter. You are the check.

That is why, with a chatbot, these failures are annoying rather than dangerous.

Now recall Section 3. An agent is the *same model*, with the human taken out from between the turns. The same five patterns run **unsupervised**, at machine speed, each step feeding the next.

Which is why the order of these courses is how it is: **you need to learn to evaluate before you learn to delegate**.

6. The Eval Mindset

AI 201 gave you techniques: self-review, "prove it", source grounding, the verification checklist. Most people use them as a **final read-through**: read the output, it seems reasonable, approve.

That is **reviewing**: pattern-matching for plausibility. And the polish trap is built to beat it.

A reviewer asks: *"Does this look right?"*

An evaluator asks: *"How would I know if this were wrong?"*

An **eval** is your answer to the second question, **written down before you look at the output**.

6. The Eval Mindset

Write the check before you see the answer

Why before: once you have read a fluent answer, it becomes the anchor. You are now checking whether it is *consistent with itself*, not whether it is *correct*. (Same reason a scientist writes the hypothesis down before running the experiment.)

Example of what it looks like at chatbot scale: three to seven lines, in a note you keep.

Before I accept the vendor comparison:

- The three vendors are the three I named
- Every number traces to the source table (not to "the analysis")
- The risk caveat survived the shortening
- It answers "which fits our constraints", not "compare them"
- I would be fine if the client forwarded this

If any check fails → go back to the step where it broke.

Do not fix forward in the same chat.

And yes, paste the same list into the prompt too (201's built-in verification). Do it. But the model checking itself is **not your eval**. It is a first filter, at best.

6. The Eval Mindset

A check is only as good as its distance from the thing it checks

From weakest to strongest:

1. **Same chat, "check your work".** It re-reads its own reasoning, with the same context and the same premises. In a long thread, it will tell you that you're absolutely right.
2. **Fresh chat, no history.** Paste the output as if it came from a stranger. Give it a critical role: "Assume there are errors in this. Find them." It has none of the conversation's assumptions. (This is "Parallelize" from Section 2, used as a critic).
3. **A different model.** Different training, different blind spots.
4. **A different source.** The original document, a calculation you do yourself, a search, a number you already know. The model isn't involved at all.
5. **A person** with the expertise you don't have.

Rule of thumb: the more it matters, the further up the numbers you go.

6. The Eval Mindset

Where to spend your checking

Don't check evenly. Check where errors hide and where you can't feel them:

- **Specifics over generalities.** Numbers, names, dates, quotes, citations, legal or technical particulars. Generic prose is rarely "wrong"; specifics are where fabrication lives.
- **Outside your expertise over inside it.** Inside, you will catch it by reading. Outside, you need a source.
- **Early in a chain over late.** Verify the extraction, not the summary (error laundering).
- **Anything that leaves under your name.** Sent, forwarded, presented, decided on.

And one question that sets the depth of the whole check:

"What is the worst thing that happens if this is wrong, and can I undo it?"

Cheap and reversible: skim. Expensive or irreversible: the full ladder, every time, regardless of how good it looks.

6. The Eval Mindset

Test the test

A check you have never seen fail is a check you don't know works.

Plant an error. Take a good output, change one number or invent one citation, and run your check (or your fresh-chat critic) on it. If it passes, your eval has a hole. Fix the eval, not just the output.

Keep informal score. Which of the five patterns did you catch this week, which did you miss, and on which kind of task (extraction, synthesis, analysis, generation, validation from Section 4)?

After a month you have something nobody can give you: a map of **how your work fails**.

6. The Eval Mindset

Your judgment, written down

For each recurring task, keep a small eval next to the prompt: three to seven checks, the ladder step it needs, and the worst-case answer.

In 201 you built a **prompt library**. This is its other half. The prompt says **what to do**. The eval says **what must be true**.

This is the point of 301. Your **taste** and your **judgment** are now written down, so they no longer have to live only in your head. A colleague can run your eval. A fresh chat can run it. And when you get there, so can something that works without you between the turns. It is the only reason you will ever be able to let it.

AI 201: *Your LLM is only as good as your ability to direct it.*

AI 301: *Your output is only as trustworthy as your ability to check it.*

Building the foundations

With time, models will improve, hallucinations will reduce, hand-holding will be less needed, guardrails and checks less necessary, and we will trust the human + AI agent system more.

But in the meantime, we continue to build the foundation of how to work together with AI in an effective and productive way, by understanding the fundamentals of what they are and what they enable

